



Agile: Product Backlog & Definition of Done

**Smart Food Analyzer: Nutrient Profiling from Food Images
(Focusing on Pakistani Cuisine)**

***Bachelors of Science in Software Engineering
(2022-2026)***

By

FIZA KHAN BSEF22M011

HUDA IJAZ BSEF22M017

ALISHBA UMER FAROOQ BSEF22M018

Supervised by:

DR SHUJA-UR-REHMAN BAIG,

Assistant Professor

DR MUHAMMAD FAROOQ,

Assistant Professor

**FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY,
UNIVERSITY OF THE PUNJAB, LAHORE.**



Contents

1. Product Vision.....	2
2. Defined Epics & Features.....	2
3. Initial Product Backlog (Stories with Priority)	2
4. Definition of Done (DoD) Checklist	3
5. Git Repo & Branching Strategy	4
6. Jira Board Screenshot	6



1. Product Vision

Smart Food Analyzer seeks to make it easier to know about food nutrition through image-based identification. The application will enable users to capture or upload images of their food and get comprehensive nutrient data, particularly on Pakistani food.

The product is meant for health-conscious individuals, fitness lovers, patients who need to follow diets, and even nutritionists who need proper and timely information on food content.

Our aim is to make nutritional consciousness more local and accessible. With deep learning combined with a filtered database of Pakistani foods, the Smart Food Analyzer will detect food items, estimate serving sizes, and compute calories and macro-nutrients. With such an approach, healthy eating is encouraged as well as the creation of technology that bridges cultural food diversity with AI.

2. Defined Epics & Features

Here are the important epics and corresponding features for the Smart Food Analyzer system:

- Epic 1: User Management
 - Secure registration and login of users
 - Profile creation with dietary preference
 - View history of analyzed meals
- Epic 2: Food Image Recognition
 - Capture or upload food images
 - Recognize and classify multiple Pakistani dishes using trained CNN models
 - Recognize mixed or multi-item plates
- Epic 3: Nutrient Profiling
 - Estimate macronutrients (carbs, protein, fat) and calories
 - Provide a comprehensive nutrient breakdown for each food item identified
 - Offer portion-based output (e.g., half plate, one serving)
- Epic 4: Portion Estimation
 - Utilize reference objects for size estimation (e.g., spoon, plate, hand)
 - Manually adjust portion count for precision
 - Save portion information to enhance model learning
- Epic 5: Analytics & Insights
 - Provide personalized daily or weekly nutrition reports
 - Plot calorie intake trends through graphs
 - Offer health recommendations based on meal patterns.



- Epic 6: Admin & Dataset Management
 - Manage dataset entries, annotations, and labeling
 - Update and retrain the recognition model periodically
 - Validate dataset quality and remove duplicates

3. Initial Product Backlog (Stories with Priority)

Priority	User Story
Must	As a user, I want to create an account so I can save my analyzed meals.
Must	As a user, I want to upload a food image so the system can detect and classify dishes.
Must	As a developer, I would like to preprocess the dataset so images are consistent for training.
Should	As a user, I would like a nutrient summary returned after every analysis so that I can monitor my intake.
Should	As a nutritionist, I would like the system to create daily reports that users can check eating habits.
Could	As a user, I would like a dark mode feature so that the interface is more visually friendly.
Could	As an admin, I'd like to periodically retrain the model with new labeled data to ensure accuracy.
Won't (for now)	As a user, I'd like voice-enabled food analysis so that I can use the app hands-free.



4. Definition of Done (DoD) Checklist

Criteria	Description	Status / Practice
Code Implemented and Committed	The feature assigned is completely developed, tested on the local machine, and committed to a dedicated feature branch as per the Git workflow.	Adopted — all work committed via feature branches.
Peer Review (Pull Request)	At least one team member reviews each branch prior to merging into the develop branch to check for code quality and maintainability.	In Practice — PRs reviewed on GitHub prior to merge.
Code Quality Standards	Code adheres to PEP8 standards, descriptive variable naming, and consists of present inline comments.	Adopted — uniform throughout scripts and notebooks.
Unit and Integration Testing	Separate testing of each module (preprocessing, model, inference) is performed to ensure reliability.	In Progress — preliminary tests developed for model training.
Build Verification (CI/CD)	Jenkins or GitHub Actions is utilized to automatically test and build the project to avoid faulty code.	Planned — integration of CI/CD in progress.
Deployment Verification	Features are deployed in a test environment and tested for expected results.	Partial — local tests performed prior to release.
Documentation Updated	README and feature-specific comments are updated to note new changes or dependencies.	Implemented — frequently updated.
Dataset Validation	Datasets are validated for correct format, labeling accuracy, and integrity.	Implemented — checked upon dataset setup.
Acceptance Criteria Met	Each feature meets the conditions outlined in its associated user story.	Practiced — tested per sprint.
Task Marked Complete in Agile Tool	Jira/Trello board is marked as completed once all above conditions are satisfied.	Implemented.

5. Git Repo & Branching Strategy

With a development branch for integrated development, a protected main branch with production-ready code, and short-lived feature/branch for new work, the Smart Food Analyzer project leverages GitHub for source control. Following code review and continuous integration checks, all changes are combined into Pull Requests (PRs). The repository structure (folders, READMEs, and .gitignore) was created in the first commits to facilitate collaborative development, and supervisors are added as collaborators with full access.



Repository Folder Structure:

The Smart Food Analyzer repository is organized for clarity and collaborative development. The folder structure separates **model training notebooks**, **datasets**, **code modules**, and **documentation** for easy navigation and version control. Only lightweight sample data is committed to GitHub, while large datasets and model files are excluded through .gitignore



Structure Overview:

Smart Food Analyzer/

- notebooks/ -> Training & evaluation notebooks
- data/ -> Sample data + external dataset link
- models/ -> Model weights & checkpoints
- src/ -> Core source code for preprocessing, inference, utils
- docs/ -> Documentation, reports, and BPMN diagrams
- requirements.txt, README.md, .gitignore

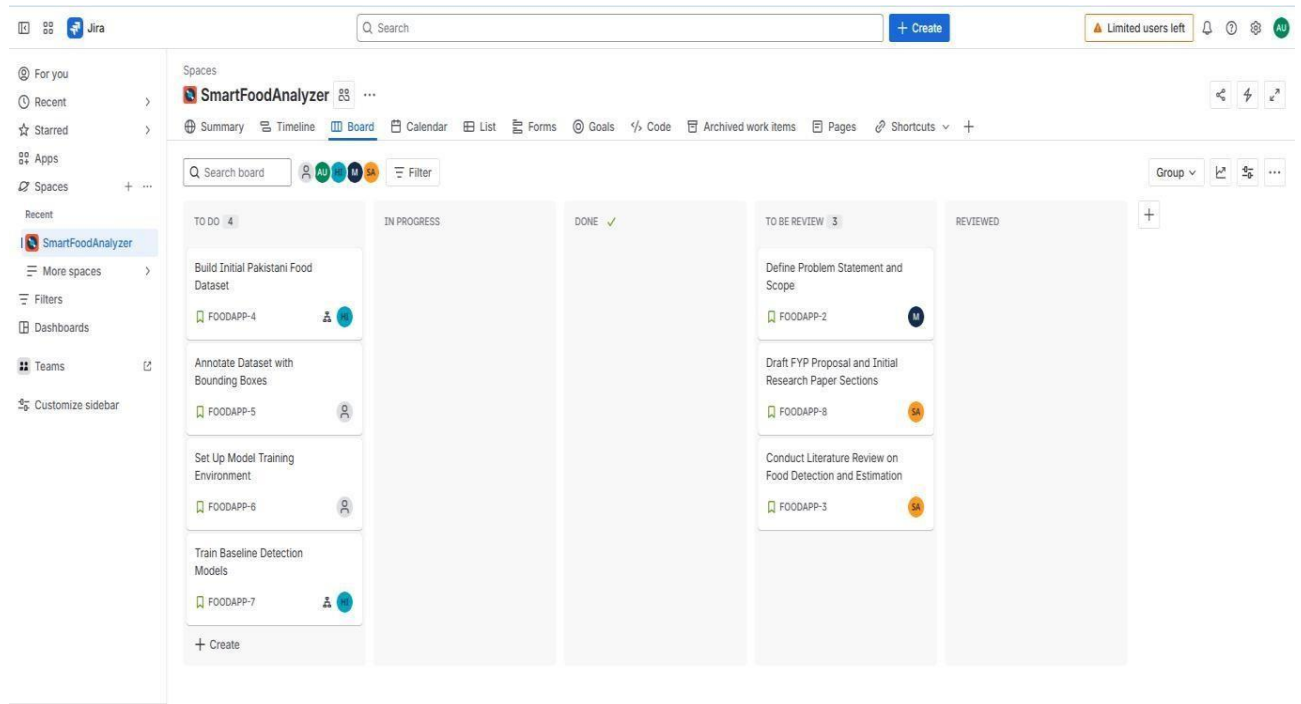
Branch model :

- **main (production)**
 - ✓ Protected. Only stable, tested releases exist here. Releases are tagged (e.g., v1.0.0).
 - ✓ No direct pushes permitted (all merges via PR).
- **develop (integration)**
 - ✓ Latest integrated features; the staging area before release. Feature branches are merged here.
- **Feature branches**
 - ✓ Naming: feature/<short-description> (e.g., feature/model-training, feature/image-upload).
 - ✓ Created from develop. Work and commits happen here. When ready, open a PR to develop.
- **Hotfix branches**
 - ✓ Naming: hotfix/<short-description> (e.g., hotfix/fix-inference-bug).
 - ✓ Created from main when urgent production fixes are required. After merge, hotfix must be merged back into develop.



6. Jira Board Screenshot

Jira Board:





Team Workload Summary:

